COMP 3000 – Artemis IDS Project Report

By Samuel Stevens

GitHub: https://github.com/Samuel-Stevens/COMP3000-Project/tree/main

Contents

Acknowledgement

Thank you to Shaymaa Al-Jubouri for constant support throughout my three years of study.

Thank you to Vivian Hocking, for his infectious laugh and surprisingly interesting policy lectures.

Abstract

With the global continuous increase in network traffic, the amount of data a network must handle as spiked significantly with Cloudflare reporting an 19% increase in traffic in 2025, as the amount of traffic grows, the rate of cyber-crime rises as well.

One of the tools often used to help detect cyber-crime is the Network Intrusion Detection System (NIDS), this is traditionally done through the use of signature recognition, matching attack signatures to network traffic.

The main down fall of this method is its inability to detect attacks it hasn't got signatures for; therefore, this report sets out to determine if Machine Learning (ML) can be used to detect attacks that the signature-based systems would miss. In this project, several ML algorithms were trained against a dataset that contains modern attack types, and the results were evaluated to determine which model would best fit use in a NIDS.

A prototype system was developed to determine its effectiveness as a tool with the tool covering data ingestion, prediction, and output to both text file and email alerts. The results of the testing showed that the Random Forest model achieved the highest score with an accuracy of 90%, this shows that ML shows promise in intrusion detection with high detection rates, which would allow for the creation of an effective NIDS.

Introduction

Project background

The growth in internet connected devices has led to a massive increase in network traffic globally, Cloudflare reports that internet traffic increased by roughly 19% in 2025(Cloudflare, 2026). As the amount of devices continues to grow along with the increase in the number of online services, networks are needing to expand to deal with the growing amount of traffic that it is facing, this growth also introduces a handful of network security problems as the higher volume of traffic and the variety makes more angles for malicious actors to try and access the network.

The more traditional form of a Network Intrusion Detection System (NIDS) is signature-based where traffic is compared against a database of rules or signatures which makes it an effective tool against established threats. This gives them a high precision against known threats and has a low false positive rate but suffers against new types of attacks as the database needs to be manually updated to deal with new threats.

To help deal with this, this project centres around the development of a Network intrusion detection system that makes use of machine learning to identify suspicious patterns within network traffic, by training against features from network traffic datasets, it can learn behaviour patterns from both normal and malicious traffic and can improve overall detection capabilities allowing the system to identify anomalies or threats that would be captured by the rule based systems.

Context

Within the current cybersecurity landscape, intrusion detection play a vital role by identifying potentially malicious traffic within a network. Network Intrusion Detection Systems (NIDS) monitor network traffic and analyses and learns patterns to help detect unauthorized access or suspicious activity. Many of these systems rely on signature-based detection, with the traffic being compared against a list of known attack signatures.

Some of the most widely used tools make use of signature-based detection, examples of these being Snort and Suricata, both of which are open source and widely used by private users and organizations. These tools are very good at detecting attacks that have been previously documented, and the system has been updated to detect.

As cybercrime grows ever more profitable and with the incident rate of cyber-crime rising by 37% over the past five years (Bridewell, 2026), the different attack types and vectors continue to evolve, with attackers modifying previously used techniques or developing new attacks using exploits not commonly known, meaning that the signature-based system can’t be updated to detect these attacks. This creates big problems for signature-based systems as attacks that haven’t yet been seen may bypass the detection systems.

Alongside the new and changing threats, modern networks generate and receive immensely large volumes of traffic due to the recent growth in cloud services, online services, and personal devices. Monitoring such a large amount of traffic data is unfeasible for an analyst to manage in a timely manner that would allow them to respond to an attack and the signature-based systems may also struggle with the highly complex and widely differing traffic patterns.

Recent research studies have explored the use of machine learning in order to enhance security detection such as the article Machine Learning for Network Intrusion Detection-A Comparative Study (Al Lail, Garcia and Olivo, 2023) in the study they implemented and evaluated a number of machine learning algorithms against the CICIDS-2017 dataset and were able to achieve detection rates of 97%. This article helps to demonstrate that machine learning models can analyse large datasets of network traffic and identify both normal and malicious traffic with very high accuracy. And unlike pure signature-based detection, has the potential to identify new attacks by recognising anomalous patterns of network activity.

With this context established, this project focuses on the development of a machine learning based Network Intrusion Detection System that can analyse network traffic data and identify and score suspicious behaviour based on its perceived risk. This aims to demonstrate the use of machine learning in network security and test the effectiveness of it when trained on a machine with modern day middle ground specifications.

Aims and Objectives

My aim for this project is to develop a machine learning based network intrusion detection system that can analyse network traffic logs to identify and classify activity that it classifies as suspicious. The aim of developing this tool is to offer improvement over the more traditional signature-based systems by making use of machine learning to detect patterns and anomalies within the network traffic.

In order to achieve my aim, my project will tackle the following objectives.

1. **Investigate pre-existing intrusion detection approaches,** notably signature-based and anomaly-based intrusion detection systems and discuss the strengths and weaknesses.
2. **Examine current enterprise and research examples of machine learning in intrusion detection,** Including commonly used algorithms and their effectiveness in detection.
3. **Select an appropriate NIDS training and testing dataset and preprocess as needed,** to ensure features aren't leaked to bias the machine learning models.
4. **Design and implement multiple machine learning models that can classify network traffic as malicious or normal.**
5. **Evaluate the performance of the models that were implemented using standard metrics such as accuracy, precision, recall, and F1 score.**
6. **Develop a prototype intrusion detection system that can process network data and provide output classifying the data into malicious and normal traffic.**
7. **Implement an alert method such as email notification, to notify the user when suspicious activity has been detected.**
8. **Critically evaluate the systems effectiveness and its limitations, including its performance, how well it would scale, and its suitability for live deployment.**
9. **To propose recommendations on how it could be improved in future.**

Literature Review

What is an Intrusion Detection System?

An intrusion detection system (IDS) is a common tool in cybersecurity that monitors network traffic or system activity for malicious activity, suspicious activity, or actions that violate the security policy of the organisation.

Where IDS splits into its sub-types is where and what they monitor, with the two main types being Network-based intrusion detection and Host-based intrusion detection.

Network-based which is the focus of this project monitors the network traffic within the network it is placed in works by analysing the packets to detect suspicious activity, such as port scanning, or malware propagation.

By monitoring inbound and outbound packets, it allows the system to observe and identify traffic patterns and anomalies across all devices on the network, making it best for identifying external attacks, large network scans, and attempts at unauthorized access.

But while it has the strength of being able to cover many hosts with a single installation, it struggles against encrypted traffic as it cannot scan for payloads and signatures within the packet.

This contrasts to Host-based which sits on individual devices and monitors system logs, file system integrity, any running processes, and changes to registry and any system calls made. By sitting directly on the device, it can detect changes to files and attempts at privilege escalation, making it very good at protecting critical devices and tracking user activity, which can help identify insider threats. But this type of monitoring can be very resource intensive as it must be deployed on every device on the network.

Despite which type of IDS is being used, they tend to fall into two types for detection, signature based which compares traffic to known attack signatures and patterns, and anomaly-based detection which works by establishing a known baseline for activity and produces alerts on deviations from this established norm.

It is worth noting that both have their own pros and cons, with signature-based having high accuracy and low false positive but misses new attack types, whereas anomaly-based tends to have slightly lower accuracy and higher false positives but makes up for this by detecting newer attack patterns.

Existing implementation of AI NIDS

There are several already existing applications that make use of AI for network intrusion detection, and example of this usage is Suricata, Suricata makes use of isolation forest models for its threat detection, allowing it to run alongside Suricata's signature-based detection systems, allowing them to offer an easily accessible hybrid detection system.

Another example of AI usage is from snort, one of the most used open-source intrusion detection systems, that traditionally makes use of signature-based detection for its systems, has introduced Snort_ML which uses neural networks instead of the traditional rule sets.

The fact that two of the biggest open-source IDS providers are shifting towards ML for detection, starts to show where the industry is starting to turn, favouring hybrid or AI systems rather than just having signature-based systems.

Legal, Social, Ethical, Professional(LSEP)

There is a number of legal problems that AI faces, one of the most common is gaining proper access to the data needed to train the model, within the AI IDS field there are 2 types of data that is used to train the model, synthetic and real, synthetic tends to be the most common type of training data available, an example of this is the UNSW NB-15, that is generally only available for usage in academic environments due to the licensing, meaning that commercially that you aren't legally allowed to use it without direct permission from the researchers that developed it.

While traditionally there aren't any social considerations relating to AI usage in intrusion detection, in the current day and age, there is a stigma around AI usage that stems from a lack of understanding of what the AI would be doing as when they hear that AI has been integrated, the mind jumps to use of Generative AI which people have a strong stance against the usage of.

The only ethical consideration for this project that I can think is how the data for training has been obtained, have I gathered it in a shady manner, through not being clear on my intentions in the gathering of data. No, in this project the data has been ethically sourced through the University of New South Wales as they provide the UNSW NB-15 dataset for free for academic usage which this project falls under.

For professional considerations, when AI is introduced into a field, there is one question that is frequently asked, will this affect jobs, for this type of implementation, no it will not, as it still requires the analyst to view this data then act on it, the same as the traditional signature-based detection systems.

This is doubly so due to its nature as an Intrusion Detection system, rather than an Intrusion Prevention System, as by itself, an IDS system can do nothing about the intrusion attempts but alert an analyst or user.

Project Methodology and System Design

Agile Development Methodology

For this project, I will make use of the Agile Development Methodology, this is a commonly used for software development in professional settings, it works by splitting work into smaller chunks that are completed over a short period of time, usually two weeks. It is split up into six stages; initiation, planning, development, testing, deployment, and review, these stages are repeating in each two-week sprint, allowing for meaningful, incremental development.

Implementation Method

I plan to implement agile in what is common practice in software development, making use of the stages described above, repeating every 2 weeks with the start and end marked by a meeting with my project supervisor. These bi-weekly meetings with my supervisor are key as it helps provide accountability and encouragement to get the work done well and on time, towards the end of this report there will be a log of my sprint notes, to help provide evidence of the work being done on time and frequently across the sprints and the project as a whole.

Alternative Approaches

While I have chosen agile for my methodology of choice for this project, there are several alternative methodologies that I could have made use of for this project but for reasons that I will detail here, were not chosen over agile.

Waterfall remains a choice often mentioned when asked about project methodologies, it fits best for projects where there are fixed requirements that are well defined, but its main weakness is that it is highly inflexible to change once development has started. Part of software development and use of technology is that problems will always appear, meaning that you may need to pivot and make changes which simply wouldn't be possible with waterfall due to its linear and rigid nature.

Spiral is a project methodology that sits in the middle between linear and iterative development, which provides it the flexibility that waterfall lacks but it still shows downsides for the type of project that I'm running as that it focuses heavily on risk management and analysis on each stage and with what I am developing, is simply not necessary, as it would only serve to bloat the development process.

The final alternative I will go into is Cleanroom software engineering, this focuses on development of software with a certifiable level of reliability, using incremental development using strong control and testing to ensure highly reliable quality code, if a piece of software fails to meet the testing requirements, testing in stopped and, you start again at the design phase.

These examples show that while there are possible alternatives to the agile methodology that ive chosen, they don't fit the project as well as they all produce either a rigidness that will cripple the project if problems occur or would add an unreasonable level of bloat that would slow the project down unnecessarily.

System Architecture

My design for the system will work along the lines of standard machine learning practice, using the standard stages of a machine learning pipeline: loading the dataset, preprocessing, feature scaling, feed the data to the model, make predictions against the data, then send an alert regarding the information. Below is a breakdown of the stages of my ML pipeline.

The first stage is loading the dataset into the Jupyter notebook file with NumPy and pandas, before loading the dataset, checking the dataset contains all of the expected features is advised considering the number of sites where these testing datasets are available from.

Next is preprocessing the data to ensure that the model can use it to train, this can involve cleaning the data to get rid of any inaccurate, incomplete, or incorrectly formatted data from

the dataset, encoding the data to convert categorical data such as text or any non-numerical categories into numerical form so that the ML algorithms can process it properly.

Dataset splitting can be vital for ML development depending on the data set being used as some datasets come split into testing and train files, this can be done in the Jupyter notebook file if needed to provide testing and training sets from a single dataset file.

Feature scaling helps ensure that features that have wider ranging magnitudes don't bias the learning process by limiting the range between a given range for example between minus 1 and 1 to reduce bias.

Loading the ML involves either training the model from scratch or loading a trained model from a saved file then feeding it the data you want it to train it on or test it against, this stage needs to be done correctly or the results will not be accurate, this stage also helps to determine issues with the previous stages as the accuracies of the models can point to problems, such as if the accuracy of the models is 99.99% it is highly likely that the models are cheating via a feature leak that is showing the models what is right and wrong.

The prediction stage is where the data is fed to the loaded and trained model and it runs against the dataset and produces predictions that can be output to a file, and then depending on the results of the predictions can trigger alerts via multiple means such as SIEM platforms, application notifications, or emails through email API's or smtp code.

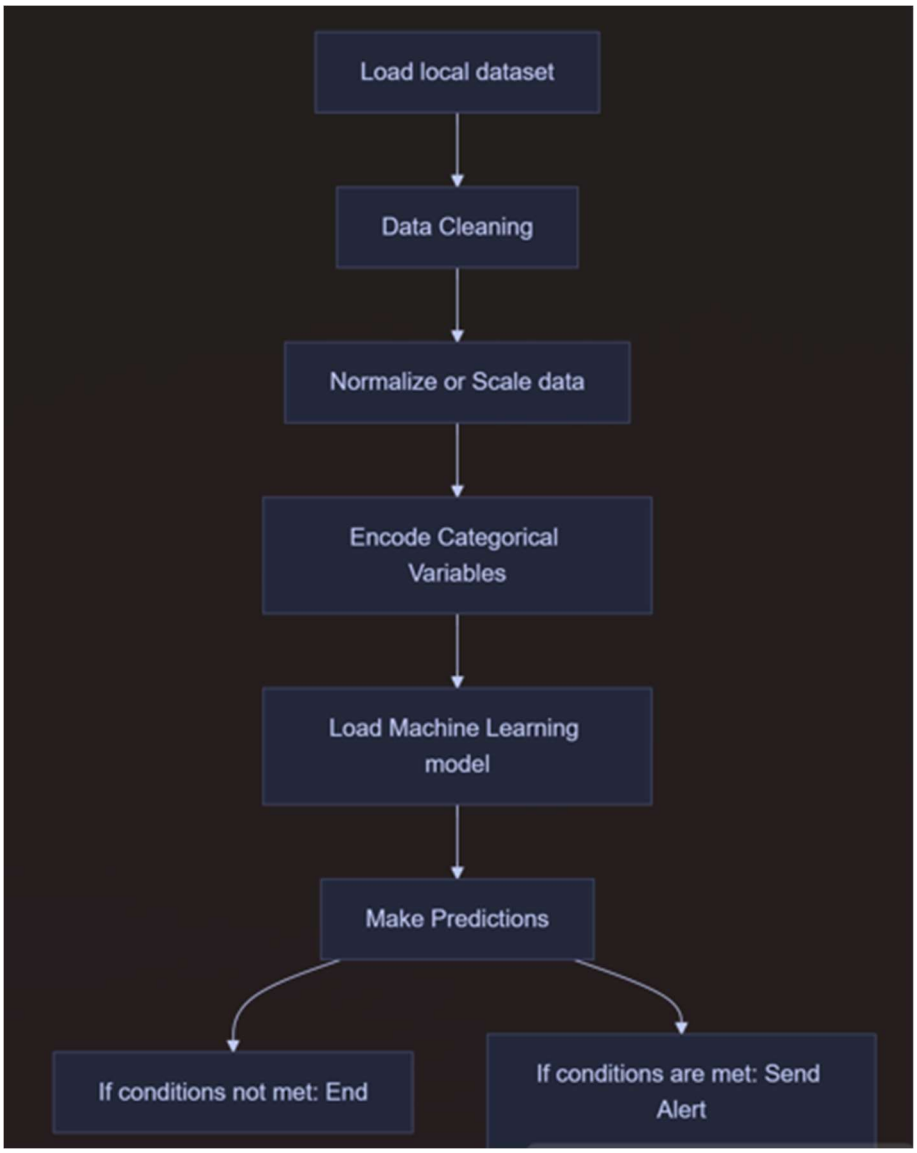


Figure 1: Pipeline diagram for the ML system.

Dataset

The dataset that this project will make use of is the UNSW-NB15 dataset produced by the university of New South Wales. This data set has forty-nine features to give the model a range of different data points to train against and learn the patterns of. It contains nine different types of attacks, these attacks are Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode and Worms.

The reason that I chose this dataset is that while it is a more modern dataset with newer types of attacks than a lot of other datasets, the hardware needed to train a model on it is much more reasonable than other modern datasets such as the CICIDS2017 dataset as part of my reason for choosing this project is to find out if AI NIDS is something that an average consumer could reasonably make use without having massive computing power to train the models.

Data Preprocessing

In order to ensure that the ML model to accept and process the data, the data must be pre-processed, it needs to go through a number of steps to that no errors occur during training and

testing. One of the most important preprocessing steps is proper data encoding, this entails changing the data type of all non-numerical categorical columns to a number format such as float64 as many ML models will not accept non-numerical data.

During preprocessing, it is key to drop columns that could poison training, for the UNSW NB15, this would be the label, id, and attack_cat as label and attack_cat denote whether a line of traffic is an attack or not, while the id column can cause models to focus on it and decrease the accuracy heavily.

A key step of preprocessing is cleaning and scaling the data, cleaning matters as it gets rid of any incomplete, incorrect, or corrupt data so that the model doesn't try to learn or make predictions based on bad data.

Scaling is used to ensure that features with large ranges don't dominate learning due to their magnitude, scaling limits to the range of numbers of features can contain, traditionally this centres around zero with a difference of one either side so minus one and positive one.

Machine Learning Models Used

In order to ensure that the model with the highest accuracy is used for final predictions, I will train and test five different ML models to determine which has the highest accuracy, this will be determined by four different metrics, the accuracy, the recall, the precision, and the f1. To clarify the metrics to help interpretation of the results further on, below are bullet points that explain what each metric is.

- **Accuracy:** Accuracy determines how many predictions the model made are true.
- **Precision:** Precision determines how many of the positive predictions made were true.
- **Recall:** How many of the data points that should be predicted as positive did we predict as positive.
- **F1:** This metric combines Precision and Recall to help determine the balance between the two and punishes heavily for heavily negative results from either metric.

The models that will be used are: Random Forest, KNN, Logistic Regression, Linear SVM, and XGBoost, these models were chosen to ensure a wider range of results by using a range of models. Below is a set of bullet points to introduce with more detail the models that will be used.

- **Random Forest:** Random Forest makes predictions through the use of multiple decision trees in order to determine a single result, making use of feature randomness to ensure low levels of correlation between the different trees.
- KNN: This algorithm works on the principle that data points that have similar traits to each other sit near each other; this is done by calculating the distances between different data points compared to unlabelled data points.
- **Logistic Regression:** This model works to predict the probability of a specific outcome occurring, in this case, whether or not a piece of data is malicious or not, the probabilities of the outcomes are confined to between zero and one, with the probabilities determined, the results are fitted to an S curve to map the values to the probability
- **Linear SVM:** This is best used for binary classification where the data can be split by a straight line into two classes, in the case for this project it would be normal and malicious traffic.

- **XGBoost:** This is a gradient boosting algorithm that works by adding more simple models to correct errors made by the models before it, it is optimized for large datasets which while and has built in routines for handling missing data.

Detection Pipeline

The pipeline for detection goes through four stages to ensure proper data flow and proper predictions, first is input where the dataset is loaded and the necessary pre-processing is completed, the data is then fed to the model and it runs predictions based on its training, the events the model marks as suspicious are saved to a text file, if the model finds suspicious events, an alert is triggered which sends an email to pre-determined email address, alerting them of the results.

Tools Used and Technologies

For this project to work, I've had to make use of several tools and technologies to fulfil my aims and objectives. I will cover the different tools I used and how and why I made use of these tools.

Python

Python is one of the main building blocks for this project as without it, none of it would work due to the availability of some key libraries that allow me to fulfil my aims, for this project, I used Python 3 due to its previously mentioned libraries for machine learning that aren't available in other languages. All code written for this project has been done in python as the language is massively versatile.

Jupyter Notebook

Jupyter notebook was used as the main IDE for this project, due to how well it complements machine learning development with its use of cells, as the cells allow you to run individual segments of code without rerunning the entire code file, which is critical for machine learning due to how long it can take to run certain code blocks, such as model training, which for very demanding models, can take many hours to run. It also stores results in the file so you can view the information gained at a later date without rerunning the code.

Python Libraries

For this project, a range of python libraries were used to implement the features set out in the aims and objectives earlier in the report, in the bullet points below, I will lay out how and why each library was used.

- **Pandas and NumPy**: This was used to process the data the code needed to run, such as the dataset csv file, and the data stored and processed by the code itself. Without these libraries.
- **Scikit-Learn + XGBoost**: These two libraries make up a large portion of the backbone of machine leaning development in python, with Scikit-Learn encompassing many different types of models, algorithms, preprocessing and a number of other features, this allows implementation of AI models to run much smoother than it would without it.

- **JobLib**: This was used in order to dump the model and feature columns from python to a PKL file to allow for usage within other Jupyter notebook files without having to retrain the model, saving a considerable amount of time.
- **SMTPLib**: This was used in order to implement email alerts, allowing the users to be informed if suspicious behaviour has been detected within the traffic log files.

GitHub

GitHub was utilized for version control to ensure that even if hardware or software problems occurred, a version was easily accessible to ensure easy access to software. It also helps provide evidence for work done due to it's easy to access commit history.

Implementation

Model Training

The aim of this phase of implementation was to determine which of the five chosen machine learning algorithms produced the highest results, determined by the metrics defined earlier in this report, these being Accuracy, Recall, Precision, and F1.

The first steps taken were to import the required dependencies, including Pandas, SKlearn and its numerous modules, NumPy, and XGBoost, then moved to establish the file path to the test and train files of the dataset and loading it into the notebook using pandas.

In [2]:
```
#Importing Dependencies
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, recall_score, precision_score, f1_score
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from xgboost import XGBClassifier
```

In [3]:
```
#Loading Data
train_path = r"F:\3000 stuff\UNSW_NB15_training-set.csv"
test_path  = r"F:\3000 stuff\UNSW_NB15_testing-set.csv"

df_train = pd.read_csv(train_path).drop_duplicates().reset_index(drop=True)
df_test  = pd.read_csv(test_path).drop_duplicates().reset_index(drop=True)

df_train.head()
```

Figure 2:Dependency Imports and Dataset Loading for Accuracy Testing

The next step of the development pipeline was the pre-processing of the data, this proved to provide more challenge than anticipated, leading to it taking up time over two sprints as there was a feature leak that was impacting the accuracy of two of the models negatively.

I started by removing the non-feature columns and any columns that could produce leaks that would poison the training and testing by either allowing the models to cheat using columns like attack_cat which contains the type of attack the line was or poison it and reduce the accuracy like the id column did during the development of this section.

```
# Remove non-feature columns and prevent leaks
drop_cols = ["srcip", "dstip", "attack_cat", "id"]
df_train = df_train.drop(columns=[c for c in drop_cols if c in df_train.columns], errors="ignore")
df_test  = df_test.drop(columns=[c for c in drop_cols if c in df_test.columns], errors="ignore")

y_train = df_train["label"].astype(int)
y_test  = df_test["label"].astype(int)
X_train = df_train.drop(columns=["label"])
X_test  = df_test.drop(columns=["label"])
```

Figure 3:Dropping features to prevent leaks.

After dropping the columns, pre-processing could begin properly, as there were multiple models that needed to be trained on this data, it needed to fit the criteria of all five, this first meant getting rid of any empty space in columns in both test and train, then made use of one-hot encoding to convert any object columns into data that can be read by algorithms that only accept numerical input.

With the columns encoded, the next task was to align the features of the testing and training dataset, this ensures that when the model are run against the test dataset, it doesn't throw errors, as the training dataset contains features that are not present in the test dataset, then as a last measure to ensure that the models do not break due to data being non numerical, the encoded columns datatype is set as float64.

The final step was to assign class weights to the training columns, this is due to XGBoost making use of class weights, so to ensure best results, I have included it.

```
#Data Preprocessing
num_cols = X_train.select_dtypes(include=[np.number]).columns.tolist()
for col in num_cols:
    med = X_train[col].median()
    X_train[col] = X_train[col].fillna(med).replace([np.inf, -np.inf], med)
    X_test[col]  = X_test[col].fillna(med).replace([np.inf, -np.inf], med)

cat_cols = X_train.select_dtypes(include=["object"]).columns.tolist()
X_train_enc = pd.get_dummies(X_train, columns=cat_cols, drop_first=False)
X_test_enc  = pd.get_dummies(X_test, columns=cat_cols, drop_first=False)
```

In [6]:

```
#Align train and test features
for col in set(X_train_enc.columns) - set(X_test_enc.columns):
    X_test_enc[col] = 0
X_test_enc = X_test_enc[X_train_enc.columns]

X_train_enc = X_train_enc.astype('float64')
X_test_enc  = X_test_enc.astype('float64')
```

In [7]:

```
#Class weights for XGBoost to ensure proper results
n_neg = (y_train == 0).sum()
n_pos = (y_train == 1).sum()
scale_pos_weight = n_neg / n_pos
```

Figure 4: Data Pre-processing and feature alignment.

The final stages of model training stage are training and results, training started by defining the models being used and the parameters that I was using to train the models, a limitation I faced during training was a hardware limitation of that I was using my own hardware to train these models which limited the depth and rigour of the model training.

As previously mentioned, the models I trained during this stage of development are Random Forest, KNN, Logistic Regression, Linear SVM, and XGBoost, I chose these models due to the range of techniques used by the different models in order to determine which would best suit and perform the best against the UNSW NB-15 dataset. Below is a section of the code showing the parameters that were used for training.

In [8]:

```python
#Model Training
models = {
    "Random Forest": RandomForestClassifier(
        n_estimators=200, max_depth=15, min_samples_split=10,
        class_weight='balanced', n_jobs=-1, random_state=42
    ),
    "KNN": KNeighborsClassifier(n_neighbors=5, n_jobs=-1),
    "Logistic Regression": LogisticRegression(
        class_weight='balanced', max_iter=500, n_jobs=-1, random_state=42
    ),
    "Linear SVM": LinearSVC(class_weight='balanced', max_iter=5000, random_state=42),
    "XGBoost": XGBClassifier(
        n_estimators=200, max_depth=6, learning_rate=0.1,
        subsample=0.8, colsample_bytree=0.8,
        scale_pos_weight=scale_pos_weight,
        tree_method='hist', random_state=42, n_jobs=-1
    )
}
```

Figure 5: Model definition code section.

Now with the model's setup with proper parameters, the models are fit against the training split of the dataset, and the various metrics can be gathered using the results of the model's running predictions against the testing split of the dataset. Below is a table containing the results of the predictions made by the five models.

Model	**Accuracy**	**Recall**	**Precision**	**F1**
Random Forest	0.9099	0.9683	0.8801	0.9221
XGBoost	0.9081	0.9646	0.8800	0.9203
KNN	0.7855	0.9423	0.7395	0.8287
Logistic Regression	0.7277	0.7556	0.7513	0.7534
Linear SVM	0.7184	0.8712	0.6948	0.7731

```python
results = {}
for name, model in models.items():
    model.fit(X_train_enc, y_train)
    y_pred = model.predict(X_test_enc)

    acc = accuracy_score(y_test, y_pred)
    recall = recall_score(y_test, y_pred, zero_division=0)
    precision = precision_score(y_test, y_pred, zero_division=0)
    f1 = f1_score(y_test, y_pred, zero_division=0)

    results[name] = {
        "Accuracy": acc,
        "Recall": recall,
        "Precision": precision,
        "F1": f1
    }
```

In [9]:

```python
#Summary of results
results_sorted = dict(sorted(results.items(), key=lambda x: x[1]["Accuracy"], reverse=True))
print(f"\n{'Model':25s} | {'Accuracy':8s} | {'Recall':8s} | {'Precision':8s} | {'F1':8s}")
print("-"*70)
for name, metrics in results_sorted.items():
    print(f"{name:25s} | {metrics['Accuracy']:.4f}  | {metrics['Recall']:.4f}  | {metrics['Precision']:.4f}  | {metrics['F1']:.4f}")
```

```
Model                     | Accuracy | Recall   | Precision | F1
----------------------------------------------------------------------
Random Forest             | 0.9099  | 0.9683  | 0.8801  | 0.9221
XGBoost                   | 0.9081  | 0.9646  | 0.8800  | 0.9203
KNN                       | 0.7855  | 0.9423  | 0.7395  | 0.8287
Logistic Regression       | 0.7277  | 0.7556  | 0.7513  | 0.7534
Linear SVM                | 0.7184  | 0.8712  | 0.6948  | 0.7731
```

Figure 6: Results code and Results from predictions against testing split.

Model Evaluation

Following on from the model training, I conducted an evaluation of the models and the results of each to determine which algorithm provided the most optimal performance for use in intrusion detection.

This was done by comparing the accuracy, precision, recall, and f1 scores of each model, shown in the table above, these metrics were chosen as they provide a balanced assessment of classification performance that is widely used as standard for model training and evaluation.

The random forest model was chosen due to its high scores across all metric showing strong detection capabilities while maintaining a proper balance between false positives and false negatives in its predictions.

Model Extraction and Deployment

With the optimal model identified, I moved to a separate Jupyter notebook file to train only the Random Forest model with same parameters that were used for the first round of training.

To ensure the model trains to the same precision as it showed during first round of training the same steps of preprocessing were used, including dropping the columns to reduce the risk of feature leak to prevent the model from cheating, to ensure fairness all of the data was encoded in the same datatype as it was previously.

For use in the next step of implementation, I saved the feature columns to a list to be saved and extracted with the model. As detailed previously, the model was trained again using the same parameters, then making use of the JobLib library, the model and feature columns were dumped into two separate PKL files and saved to my PC.

The reasoning behind doing this extra step is that it allows me to reuse the models for predictions in other notebooks, without the need for retraining, significantly reducing the computation processing overhead and allows for faster predictions during operation.

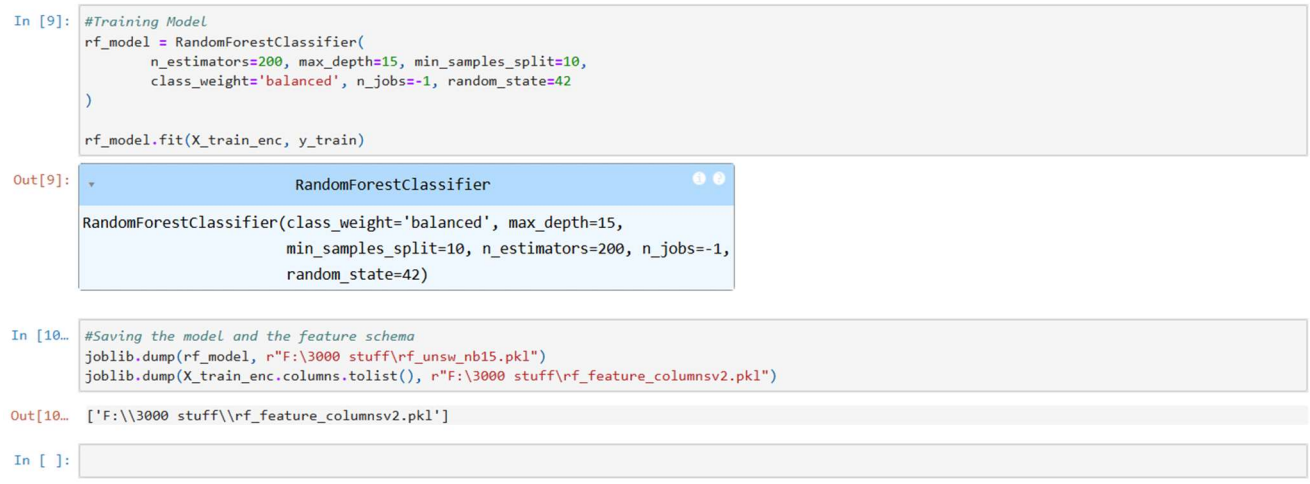


Figure 7: Output showing files successfully saved.

With the model and feature schema saved to a pkl file, the implementation of an output system for the NIDS could begin. To start, I made a new Jupyter notebook file, my reasoning for splitting the code for this project into three different files is that it makes the code easier to process, since all the code is in documents that describe what its purpose is clearly, it also reduces the computational overhead if the python kernel that it is running on needs to be reset for whatever reason, which would require all cells of the notebook to be run again, taking both time and computational resources.

As it is a new notebook file, the model needed to be loaded, as it had been saved, I loaded the PKL file of the Random Forest model trained in the previous notebook, after loading the feature schema also saved previously, I moved to pre-process the data using the same methods used previously, with the added step of feature alignment to ensure that the model does not throw errors since the test split is missing some columns from train split.

With data pre-processed, the model can make predictions against data which is saved to be used by the output system and the email alert system that will be covered in the next two sections.

```
#Feature alignment to ensure there isnt mismatch between the columns of test and training

X_test_enc = X_test_enc.reindex(columns=feature_cols, fill_value=0)


X_test_enc = X_test_enc.astype("float64")
```

Figure 8: Implementation of Feature Alignment

NIDS Output System

The predictions ran by the model mark the rows of network traffic with a risk score between zero and one, this is the backbone of both the alert system and output system, this is because the results are filtered based on a threshold that can be adjusted manually to any number between zero and one. This dictates how strict the results are filtered, the threshold level I used during testing was 0.9, this means that any rows of data that were ranked below 0.9 risk score aren't included in output.

To establish context for output, during pre-processing, certain columns were preserved that would provide the most insight during output. The columns saved and the code used to do so can be seen below.

```
In [8]:
#Preserving certain columns in order to make text output more understandable
output_context = df_test[[ "state", "service", "proto", "attack_cat", "id" ]].copy()
```

Figure 9: Columns preserved for context for output.

The output system functions by having the model predict the probability of traffic being malicious based on its training, and saving these results to the variable “probs” the comparing the resulting probabilities to the threshold, in this case 0.9, then taking the results that are over the threshold and outputting the score along with the columns that were preserved for context to a txt file.

```
In [24]:
#Extract events marked as suspicious and then write to a TXT file
suspicious_events = results[results["suspicious"]]
output_file = r"F:\3000 stuff\log_events_marked_by_rf.txt"

with open(output_file, "w") as f:
    for _, row in suspicious_events.iterrows():
        f.write(
            f"[ID={row['id']}] "
            f"[RISK={row['risk_score']:.2f}] "
            f"PROTO={row['proto']} SERVICE={row['service']} "
            f"ATTACK={row['attack_cat']}\n"
)

print(f"Data Extraction Complete. {len(suspicious_events)} suspicous events written to:")
print(output_file)
send_email_alert(suspicious_events, threshold)
```

```
Data Extraction Complete. 37484 suspicous events written to:
F:\3000 stuff\log_events_marked_by_rf.txt
Email sent successfully
```

Figure 10: Code for outputting suspicious code to a .TXT file.

Email Alert System

The email alert system makes use of the same backbone that the TXT output system makes use of, namely the risk score, but instead of outputting through text, this system makes use of the SMTPLib library to function and googles smtp server to send the email.

To implement this, I had to make an email address for the system and setup and app password to allow the code to send emails automatically when the code is run.

After setting the credentials to be used by the system, the next step was to implement the function that would gather the results, compose the email, and send the alert. This started by setting up the basic email content, its subject, it’s “To”, and its “From”, with the “To” set as my personal email for the purposes of testing the prototype.

```
In [23…  #Setting up Email Credentials and Alert
sender_email = "artemisids3000@gmail.com"
sender_password = [redacted]
recipient_email = "samuelstevens2902@gmail.com"

def send_email_alert(suspicious_events, threshold):
    if suspicious_events.empty:
        print("No high-risk event. No Email has been sent.")
        return
    #Set up email content
    msg = EmailMessage()
    msg["Subject"] = f"Artemis Alert: {len(suspicious_events)} High-risk events discovered"
    msg["From"] = sender_email
    msg["To"] = recipient_email
```

Figure 11: Code to configure credentials and basic contents [Password hidden for security purposes]

After this was to implement the proper body of the email, this started by establishing an alert summary variable that would display a few rows of the data in the email , to show the analyst an example of the scores the data has achieved to inspire urgent action. This was done as the test data set contains no Ip addresses limiting the sensitivity of the information of the dataset while this can be helpful in this setting, it is a definite limitation of the dataset.

After the variable has been set properly, the full body can be defined, due to the nature of this system as a prototype, I have kept the body simple and text based, a future improvement if the development of project were to continue would be to overhaul the design of the email to look more professional, below you can see a code snapshot showing the contents of the body.

```
#Make a summary of the top alerts
alert_summary = suspicious_events[["id", "proto", "service", "attack_cat", "risk_score"]]

body = f"""
Suspicious Activity has been detected

Threshold: {threshold}
Suspicious events detected: {len(suspicious_events)}

These are a number of the events that triggered this alert.
{alert_summary.to_string(index=False)}

This is an automated alert sent from Artemis IDS.
"""
msg.set_content(body)
```

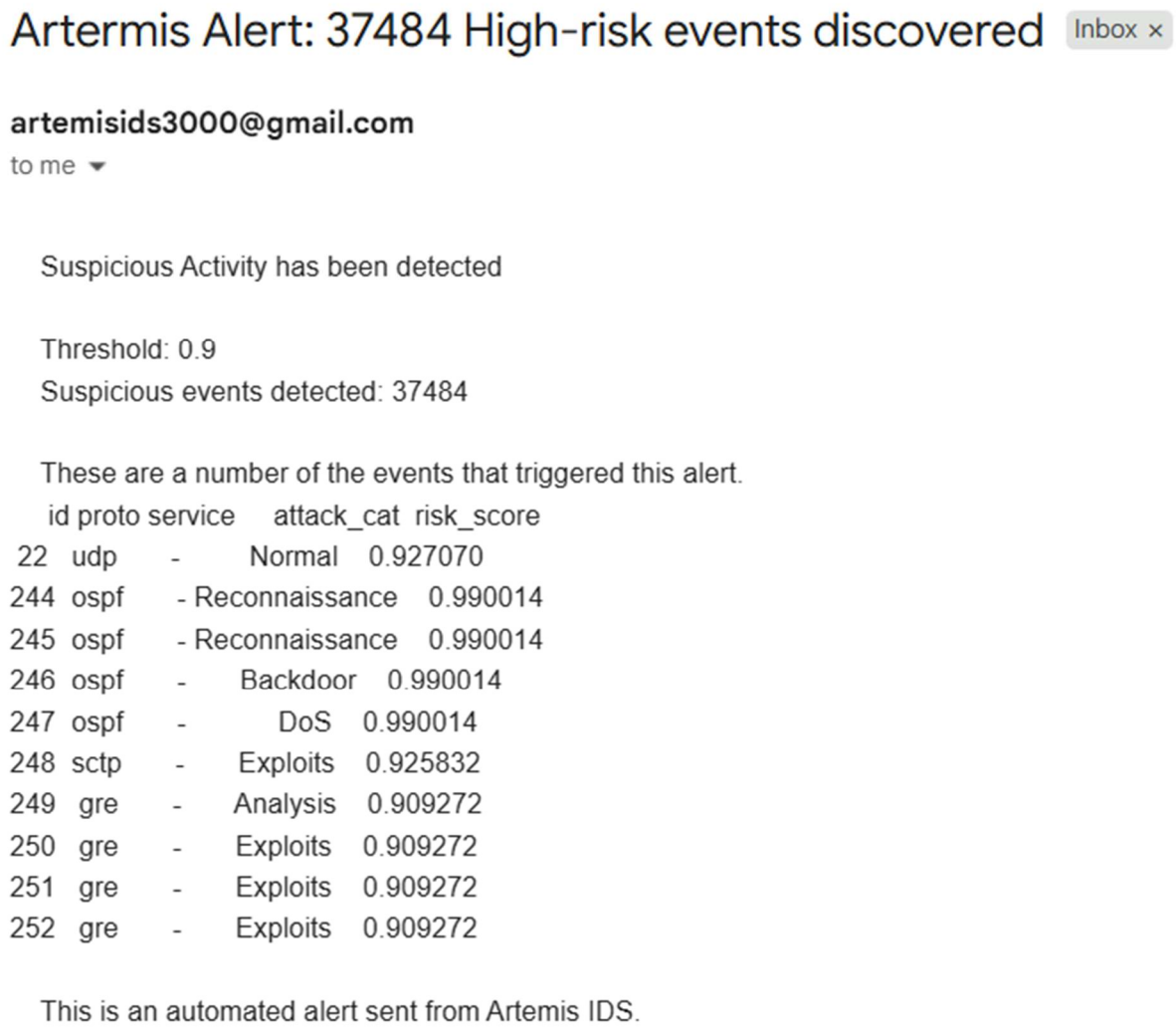
Figure 12: Content of Email body code.

With the body now set, the next step of implementation is to add the code needed to send the email, this was achieved using the SMTPlib library and its SMTP_SSL feature, making use of googles SMTP server. Below is the code that was used to achieve this.

```python
#Sending the email
try:
    with smtplib.SMTP_SSL("smtp.gmail.com", 465) as smtp:
        smtp.login(sender_email, sender_password)
        smtp.send_message(msg)
    print("Email sent successfully")
except Exception as e:
    print("Failed to send Email", e)
```

Figure 13: Code for sending an automated email through SMTP.

The main reason for implementing alerts into this system is due to its nature as an Intrusion Detection System rather than an Intrusion Prevention System, and IDS cannot act on its own to prevent system intrusion, it can only alert the analyst or user who can take action on the information. Below is an example of the output, in real world usage, the attack_cat column wouldn't be present, but it is very useful for testing and validation.



Artermis Alert: 37484 High-risk events discovered Inbox ×

artemisids3000@gmail.com

to me

Suspicious Activity has been detected

Threshold: 0.9
Suspicious events detected: 37484

These are a number of the events that triggered this alert.

	id	proto	service	attack_cat	risk_score
22		udp	-	Normal	0.927070
244		ospf	-	Reconnaissance	0.990014
245		ospf	-	Reconnaissance	0.990014
246		ospf	-	Backdoor	0.990014
247		ospf	-	DoS	0.990014
248		sctp	-	Exploits	0.925832
249		gre	-	Analysis	0.909272
250		gre	-	Exploits	0.909272
251		gre	-	Exploits	0.909272
252		gre	-	Exploits	0.909272

This is an automated alert sent from Artemis IDS.

Figure 14: Example email output from the NIDS system.

Software Engineering Principles

All code development in this project followed key software engineering principles. Code was structured into modular components to improve reusability, readability, and debugging. Reusable functions were implemented where appropriate following Don't repeat yourself (DRY) principles, and unnecessary complexity was avoided wherever possible.

Implementation sprint logs

Sprint 1 – Tools and technologies research

Objectives

Primary aim of this sprint is to research tools that will be key for developing the AI NIDS and begin work on initial model accuracy training.

Work done

A number of tools have been identified that will be key for development, these tools are Jupyter Notebook for use as the main IDE for the project, Python is to be used as the main development language for the project, SK-learn for its many modules and algorithms that make AI development so effective in python, XGBoost for its ML algorithm, Pandas and NumPy for data ingestion, and UNSW NB-15 has been chosen as the dataset for training and testing the models.

Five algorithms have been chosen to train and test the accuracies of to determine the optimal algorithm to use for the NIDS they are Random Forest, KNN, Logistic Regression, XGBoost, Linear SVM.

Issues Encountered

No issues encountered in this sprint

Solutions

No issues were encountered.

Reflection and Next steps

This sprint has been key for identifying the tools and technologies that will be key for this project's development. The next steps are to start implementing model accuracy testing.

Sprint 2 – Model accuracy testing implementation

Objectives

Develop a Jupyter notebook file to test the accuracy of the five ML algorithms.

Work done

Implementation is underway, preprocessing has been completed, models have been defined and predictions have been made but accuracies of Random Forest and XGBoost suspiciously low, around 35% – 40%, cause has not been identified yet.

Issues Encountered

Very low accuracies of certain models compared to other model such as KNN which achieved an accuracy for roughly 80%.

Solutions

No solution discovered for this issue.

Reflection and Next steps

Needed to ensure that the pro-processing is fits the needs of all five models with their different requirements, also showed the importance of dropping tables that could present problems. Tests to determine cause of problems to be undertaken during next sprint.

Sprint 3 – Solve model accuracy problems

Objectives

Resolve model accuracy problem encountered during previous sprint. Identify possible feature leakage.

Work done

Problem has been identified and sorted, the “id” column was poisoning the model training and causing the accuracy problems found during the previous sprint, accuracies of random forest and XGBoost now sit around 90%.

Issues Encountered

No issues encountered during this sprint.

Solutions

Dropped “id” column which solved the feature leakage problems that was discovered during the previous sprint.

Reflection and Next steps

The identification of columns that could let the models either cheat and increase their accuracy unnaturally and columns that will poison the training and lowering the accuracy is of utmost importance. Next steps are to train and dump then extract the model with the highest accuracy; in this case it is Random Forest to be used for predictions for output and alerts.

Sprint 4 – Model training, dumping and extraction

Objectives

The main objective of this sprint is to dump and extract the trained random forest model to allow for its reuse without requiring re-training each time a new Jupyter notebook is made.

Work done

A new Jupyter Notebook file was made for this task to allow for easier training in the future, the same parameters were used for training the random forest model which was then saved in a .PKL file, the feature schema was also saved to be used in the development of the output and alert systems.

Issues Encountered

No issues were encountered during this sprint.

Solutions

N/A

Reflection and Next steps

Extracting the model once it has been trained makes a lot of sense and wasn't something I had considered before starting this project the fact that you don't need to retrain it makes it worth it in the amount of computational power you end up saving.

Work on the output system starts next sprint, likely outputting to txt file, specifics will be defined once implementation starts and I know what I am working with.

Sprint 5 – NIDS Output System

Objectives

The main objective of this sprint is to develop an output system for the NIDS system outputting the results of the predictions to a txt file with preserved columns to give the predictions context. Results to be filtered for output by the probability gauged by the model with records over a certain percentage being included in the output.

Work done

A new notebook file was created for this system to make the system more modular and make it easier to maintain, the model that was saved last sprint was loaded into the notebook and preprocessing was done like the previous sprint but issues occurred since I was not making use of the training dataset.

There were more steps to feature alignment this time as the system started throwing errors due to column mismatch, so I had to find a work around to satisfy the requirements enough for it to work.

The model assigns probabilities to each record based on the risk it poses according to the model, these results are range between 0 and 1 and any record with a score of over 0.9 gets stored within the results output TXT file.

Issues Encountered

The system threw errors relating to feature mismatch between what the was expecting from the columns and what it got, this is due to the columns of test and train for UNSW NB-15 being slightly different, an example of this is the srcip and dstip columns which the train set has while the test does not.

Solutions

The solution for this was to double down on feature alignment using the feature cols file that was saved from the previous sprint.

Reflection and Next steps

This sprint highlighted the importance of ensuring that the columns of a dataset stay consistent between the split used to train and the split used to test, especially if they are separated into separate files from the start such is the case with the UNSW-NB15 dataset.

The next step is to implement and alert system as without a way to generate alerts, an IDS poses little help, as it can't stop intrusion only detect it.

Sprint 6 – Email Alert System

Objectives

The main objective of this sprint is to implement an email alert system that will run when predictions are made against the dataset, this is to be done using smtp.

Work done

Email alert system has been implemented and tested, implemented using the SMTPlib library and using googles smtp servers using an email address created for this purpose and an app password for credentials to avoid leaking the real account password.

Issues Encountered

No issues were encountered during this sprint.

Solutions

N/A

Reflection and Next steps

This sprint showed me how reasonable it is to code an email function and taught me how obtuse google is about allowing the creation of app passwords. The next step of implementation is to wrap up development, I have implemented all features that I consider key for the NIDS system and am happy with the state of the prototype system, Supervisor approves of winding down development.

Sprint 7 – Wrap up of development

Objectives

The main objective of this sprint is to clean up as development winds to a close, another round of testing is to be completed to double check that all accuracies are accurate, and the output both through text and email are valid. Clean up comments to ensure they are easy to read and make sense.

Work done

Testing has been completed, all systems produce the expected outputs, the accuracies from model accuracy testing are within expectations, both outputs systems output the expected outputs, there are some false positives within output but that is within expectations.

Issues Encountered

No issues were encountered during testing and validation.

Solutions

N/A

Reflection and Next steps

With development ending, it has allowed me to reflect on the whole of development and the interesting journey it has been especially as I had little prior experience with AI development outside of the AI module during second year.

The next steps of the project are to continue work outside of development, poster and report and the other tasks yet to be completed.

Testing and Validation

Testing

Testing during implementation was done as each notebook was developed, this was made easier using Jupyter notebooks cells as each cell can be run individually making testing simpler, allowing for incremental component testing throughout development.

During the implementation of the model accuracy testing notebook, testing discovered the accuracy problem that was described during the implementation section of this report.

Due to the ongoing nature of testing during the implementation, at the end of development, a round of system wide testing was conducted testing all systems including the model accuracy, model training and extraction, and the output and alert systems.

Output Validation

With how key the output is for a NIDS, making sure that the information outputted by the system is valid was key, the results were validated by including the attack_cat column in the output, the use of this column allowed analysis to see if the predictions made by the system were accurate and while there were a number of false positives within the output which while unfortunate was expected due to the nature of machine learning, the output was accurate to expectation with a high levels of true positives within the output.

While this validation process has shown the model is effective against this dataset, its performance in live settings may vary due to the variance in traffic patterns between the dataset it was trained against and the real-world traffic data.

Discussion and Critical Evaluation

System Performance

The performance of the system has met the expectations that were set when this project with two models hitting 90% accuracy during accuracy testing providing a strong base to move onto implementation of the rest of this project, this allowed for strong predictions to be used in the output and alert systems.

Random forest was chosen as the main model to be used for predictions due to its higher recall rate compared to XGBoost, this is due to recall being very important in intrusion detection systems as failing to identify malicious traffic can result in attacks going undetected, a high recall score indicates strong detection performance, with random forest scoring 0.9683 and XGBoost scoring 0.9646, making Random Forest the stronger choice for this system.

Below are a copy of the results to refresh the memory of the readers.

Model	**Accuracy**	**Recall**	**Precision**	**F1**
Random Forest	0.9099	0.9683	0.8801	0.9221
XGBoost	0.9081	0.9646	0.8800	0.9203
KNN	0.7855	0.9423	0.7395	0.8287
Logistic Regression	0.7277	0.7556	0.7513	0.7534
Linear SVM	0.7184	0.8712	0.6948	0.7731

The output of the prototype has met expectations with both text based and email output passing testing and validation,

Strengths of System

There are a number of strengths of this system, one of the main being its modularity, this allows for easy customization of models and allows for quick retraining and extraction based on the changes made, the contents of the results and alerts are also easily altered, allowing for changes in what columns are included for context, or how strict the output filtering is by changing the threshold.

Another strength of the system are its reasonable training requirements compared to some other systems which use over tuned models that use more computational power than required. The computation power needed to run this system is reduced again due to extraction of a pre-trained model meaning that you can run predictions on multiple devices while only needing to train the model once by saving it to a PKL file.

Limitations

While the system operates at the level matching the original expectations set at the start of this project and all features set out in the objectives have been implemented successfully, there is room for improvement in the system, one of the key examples of this is that it in its current form as a set of Jupyter notebook files, it isn't able to continuously run which in the real world would be a serious detriment but isn't too serious in the context of this project due to its academic nature but is a area for future improvement.

The system is also unable to capture real time traffic to work against which can be key to real world systems, this was left out intentionally due to possibility that it could be used to monitor network traffic unlawfully, while this can be achieved using publicly available tools such as Wireshark, I felt it was best to leave it out of this system.

Then there is one of the key limitations for this project, the dataset, this is described as a limitation due to its nature as a synthetic dataset, along with some feature issues I encountered during the implementation of this projects system.

Synthetic datasets while can be very useful in training in models such as its use in this project, if there is real world data available it is better to use the real data due to increased amount of edge cases that assist anomaly detections which NIDS tends to make use of.

The specific dataset used is this project has its own limitations the main limitation that I noticed during the development was the lack of IP fields in the pre-prepared testing split, though it is possible this was done to avoid bias due to IP.

The final limitation that will be mentioned applies to all ML models but can have more critical effect in the area of intrusion detection, the limitation is false results, both positive and negative results can have disastrous results in intrusion detection, with false negatives not catching malicious traffic and false positives creating more work for the analysts managing the system, leading them down a rabbit hole that distracts from actual malicious traffic.

Comparison against existing systems

The issue that arises when trying to compare AI NIDS to traditional versions is that they operate with completely different detection methods, with the AI systems working through traffic pattern anomaly detection, and the traditional systems working by detecting known signatures in the traffic patterns as has been mentioned previously in this report. Tradition systems tend to have higher overall true positives with reduced false positives due to their expansive signature databases but struggle heavily with new attacks that hasn't been added to the systems yet.

Whereas the AI systems can deal with new attacks due to their anomaly-based detection methods have, but tend to have higher false positive rates which increases the work load on analysts and shows one of the main limitations of only using ML based detection.

As a result of the drawbacks of both version, the most effective method of intrusion detection tends to be a hybrid approach making use of both signature-based and anomaly-based, which is what I would consider the most likely use of my system would be if it went into a live system, with its high recall, it would make a strong component of a hybrid detection system.

Future work

While the system has met all set objectives relating to its development, there are some features or improvements that could be made if work were to continue into the future but fell beyond the original scope of this project or would make it more suitable for a live running environment or weren't key due to the academic nature of this project.

One of the easiest improvements that could be made for this system would be to move it out of Jupyter notebook and into a more standard python file, this would allow for the system to run 24/7 provided the system it is running on doesn't go down, this would be fairly simple to implement as it could be done by designing a piece of python code to run the prediction/output

file after a certain time, this would bring the system more in line with NIDS used in live environments.

Another feature would possibly enhance the system would be the addition of a deep learning model to produce another set of predictions, though accuracy testing would be required to gauge whether its addition to the system would help or hinder accurate predictions.

A piece of future work that would allow for more thorough testing of the system would be the use of a real-world dataset but the issue is obtaining them due to the sensitive nature of the real-world traffic data.

The final piece of future work to be introduced is further work to increase the accuracy of the model, there are a couple of ways this can be done, use a more expansive dataset allowing for more thorough training, and to increase the training parameters and use more powerful hardware for training, as by increasing the parameters allows the models to explore more options, it allows random forest more trees with more depth and branches.

Evaluation against aims and objectives.

Over the duration of the project, the aims and objectives set out in the beginning of the project have been successfully completed through continuous work covering a combination of research, design, and implementation.

The initial objectives that covered background research and context relating to the usage of AI in intrusion detection were completed during the literature review, which covered both traditional and AI based intrusion detection in detail. This provided a strong backbone for the decisions relating to design and implementation later in the project.

The technical objectives set out in the beginning have been completed successfully, with all features designed and implemented, including the training and accuracy testing of multiple AI algorithms, these models where then tested using appropriate metrics namely, precision, accuracy, recall, and f1. The alert system detailed in the objectives was successfully implemented through the use of SMTPlib and googles smtp server.

The objective of evaluating the systems performance was completed through component and system testing during development, the chosen model achieved high score overall with a notable recall score, showing its effectiveness in detection.

Finally, the objectives relating to the critical evaluation of the system and the future work to improve the system has been covered in the discussion section of this report, with key limitations of the system being identified and reasonable improvements being proposed.

Overall, the aims and objectives of the project have been met through the development of an AI-based Network Intrusion Detection system, while also taking into consideration its practicality in real world deployment.

Conclusion

Over the course of this project, I have developed an AI-based intrusion detection system that is able to classify and score the probability of network traffic being malicious or benign, and developed supporting systems such as text output that allows for analysis of the data by analysts, and an email alert system that sends out email alerts when high-risk behaviour has been identified.

All aims and objectives set out during the beginning of this project have been successfully fulfilled, with all nine objectives being achieved, including the critical evaluation of the system, including its strengths and limitations, and its performance.

I found during this project that the way forward for intrusion detection is not to focus on either traditional systems or AI-systems but to introduce a hybrid systems as the two types cover the weaknesses of other with signature based having the overall higher accuracy while the AI-systems handle unknown attacks much better than the traditional systems, this allows for a much more comprehensive detection system compared to only focusing on one or the other.

Overall, this project has been a success with all goals that were established at the start of the project being met and all features fully implemented and tested with the outputs validated, the limitations identified and with areas for future improvement identified as well, makes this project an overall success.

References

Cloudflare, “Cloudflare 2025 Year in Review”. Available at https://radar.cloudflare.com/year-in-review/2025 (Accessed15/03/2026)

UK Public Cyber Crime Incidents Rise 37% Over Five Years. Available at: https://www.bridewell.com/insights/blogs/detail/uk-public-cyber-crime-incidents-rise-37--over-five-years (Accessed 16/03/2026)

Al Lail, M., Garcia, A. and Olivo, S. (2023) ‘Machine Learning for Network Intrusion Detection—A Comparative Study’, *Future Internet*, 15(7), p. 243. Available at: https://www.mdpi.com/1999-5903/15/7/243

*Moustafa, N. and Slay, J. (2015) *UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)*, *IEEE Xplore*. Available at: https://doi.org/10.1109/MilCIS.2015.7348942

*Moustafa, N. and Slay, J. (2016) ‘The evaluation of Network Anomaly Detection Systems: Statistical analysis of the UNSW-NB15 data set and the comparison with the KDD99 data set’, *Information Security Journal: A Global Perspective*, 25(1-3), pp. 18–31. Available at: https://doi.org/10.1080/19393555.2015.1125974

*Moustafa, N., Slay, J. and Creech, G. (2017) ‘Novel Geometric Area Analysis Technique for Anomaly Detection using Trapezoidal Area Estimation on Large-Scale Networks’, *IEEE Transactions on Big Data*, pp. 1–1. Available at: https://doi.org/10.1109/tbdata.2017.2715166

*Moustafa, N., Creech, G. and Slay, J. (2017) ‘Big Data Analytics for Intrusion Detection System: Statistical Decision-Making Using Finite Dirichlet Mixture Models’, *Data Analytics and Decision*

Support for Cybersecurity, pp. 127–156. Available at: https://doi.org/10.1007/978-3-319-59439-2_5

*Sarhan, M. *et al.* (2021) 'NetFlow Datasets for Machine Learning-based Network Intrusion Detection Systems', *arXiv:2011.09144 [cs]*, 371, pp. 117–135. Available at: https://doi.org/10.1007/978-3-030-72802-1_9

(*) = No citation in text, referenced here due to requirements for use of the UNSW NB-15 Dataset

Appendix

Student Declaration of AI Tool use in this Assessment

Please indicate your level of usage of generative AI for this assessment - please tick the appropriate category(s).

If the "Assisted Work" or "Partnered Work" category is selected, please expand on the usage and in which elements of the assignment the usage refers to.

Solo Work	**S1 - Generative AI tools have not been used for this assessment.**	☐
Assisted Work	**A1 – Idea Generation and Problem Exploration** Used to generate project ideas, explore different approaches to solving a problem, or suggest features for software or systems. Students must critically assess AI-generated suggestions and ensure their own intellectual contributions are central.	☐
	A2 - Planning & Structuring Projects AI may help outline the structure of reports, documentation and projects. The final structure and implementation must be the student's own work.	☐
	A3 – Code Architecture AI tools maybe used to help outline code architecture (e.g. suggesting class hierarchies or module breakdowns). The final code structure must be the student's own work.	☐

	A4 – Research Assistance Used to locate and summarise relevant articles, academic papers, technical documentation, or online resources (e.g. Stack Overflow, GitHub discussions. The interpretation and integration of research into the assignment remain the student's responsibility.	☐
	A5 - Language Refinement Used to check grammar, refine language, improve sentence structure in documentation not code. AI should be used only to provide suggestions for improvement. Students must ensure that the documentation accurately reflects the code and is technically correct.	☒
	A6 – Code Review AI tools can be used to check comments within the code and to suggest improvements to code readability, structure or syntax. AI should be used only to provide suggestions for improvement. Students must ensure that the code accurately reflects their knowledge and is technically correct.	☐
	A7 - Code Generation for Learning Purposes Used to generate example code snippets to understand syntax, explore alternative implementations, or learn new programming paradigms. Students must not submit AI-generated code as their own and must be able to explain how it works.	☐
	A8 - Technical Guidance & Debugging Support AI tools can be used to explain algorithms, programming concepts, or debugging strategies. Students may also help interpret error messages or suggest possible fixes. However, students must write, test, and debug their own code independently and understand all solutions submitted.	☒
	A9 - Testing and Validation Support AI may assist in generating test cases, validating outputs, or suggesting edge cases for software testing. Students are responsible for designing comprehensive test plans and interpreting test results.	☐
	A10 - Data Analysis and Visualization Guidance AI tools can help suggest ways to analyse datasets or visualize results (e.g. recommending chart types or statistical methods). Students must perform the analysis themselves and understand the implications of the results.	☐

	A11 - Other uses not listed above Please specify:	☐
Partnered Work	**P1 - Generative AI tool usage has been used integrally for this assessment** Students can adopt approaches that are compliant with instructions in the assessment brief. Please Specify:	☐

Please provide details of AI usage and which elements of the coursework this relates to: AI was used to support debugging efforts during development and to refine language as ive received feedback numerous times that I write too casually or in first person too much.

I understand that the ownership and responsibility for the academic integrity of this submitted assessment falls with me, the student.	☒
I confirm that all details provide above are an accurate description of how AI was used for this assessment.	☒